

IDEA DEEP-DIVE · TECHNICAL PROPOSAL

AI Secure Dev Tool

바이브 코딩(vibe coding) 시대의 보안 사고를 생성 루프 중간의 자동 보안 검증으로 예방하는 개발 도구 — 전형적 하네스를 넘어서는 기술 차별화와 개발 계획

뉴로심볼릭 정적분석

보안 지식그래프

생성 중 개입

LLM 퍼징

멀티에이전트 검증

형식검증

근거: ICLR 2025 · IEEE S&P · Google Project Zero · GitHub Engineering · arXiv 다수 | 작성일 2026-07-20

01 문제 정의 — 왜 지금인가

AI가 개발을 대신하며 코드가 파도처럼 쏟아지지만, 검증 피드백 루프가 빠져 보안 결함이 급증하고 있다. 시장 데이터가 이를 뒷받침한다.

1.7×

AI 코드가 사람 코드 대비 이슈 발생 배수 (CodeRabbit, 470 PR)

2.74×

보안 취약점 — 전 항목 중 최대 배수

~40%

Copilot 생성 코드 중 취약 비율 (Asleep at the Keyboard, S&P '22)

+41%

Copilot 도입 팀의 버그율 증가 (Uplevel, 800 dev)

핵심 진단: 빠진 것은 "검증 루프"

사람은 로컬 실행 → UI 테스트 → 테스트 작성을 거친 뒤 푸시한다. AI 워크플로는 이 단계를 건너뛰고 생성 → 리뷰 → CI로 직행한다. 이 "손으로 확인하는 단계"의 소실이 버그 배수를 만든다.

출처: Shiplight, *AI-Generated Code Has More Bugs*

본 도구의 명제

생성 루프의 중간에 "자발적 개발 보안" 단계를 자동 삽입한다. 프로젝트 유형 (웹/앱/SW·프레임워크)에 따라 맞춤 보안 프로토콜을 동적으로 적용한다.

시장 관찰과 설계 방향이 정확히 일치 → 문제 정의가 견고함.

02 시장 공백 — 우리의 진입 지점

기능 QA 자동화는 이미 경쟁자가 있으나, 보안 검증 루프는 여전히 "사람이 하라"고 남겨져 있다.

Shiplight (인접 경쟁자) = 기능 QA

브라우저를 열어 앱이 동작하는지 검증(에이전틱 E2E, MCP·YAML). 그리고 명시적으로 선을 긋는다:

"자동화는 기능적 정확성을, 보안·위협 모델링·컴플라이언스는 사람 리뷰어가 맡아라."

본 도구 = 보안 QA (남겨진 칸)

그들이 사람에게 떠넘긴 보안 검증 루프를 자동화한다. 경쟁이 아니라 상보 관계.

흐처를 원칙: "PR마다가 아니라 AI가 만든 diff마다 재검증" → 보안에 적용하면 diff-scoped 보안 게이트.

근거의 질 주의: CodeRabbit·Shiplight 자체 케이스스터디는 벤더/마케팅성. 동기부여엔 좋으나, 신뢰도 근거로는 피어리뷰 자료(S&P·ICLR)와 반드시 병기할 것. GitClear의 churn·중복 지표는 보안이 아니라 유지보수성 근거로 구분해 사용.

03 "전형적"인 부분 vs 차별화 포인트

"짜고 → 스캔하고 → 고친다"는 이미 베이스라인이다. 아래 6개 기술로 그 위를 넘어서는다.

▲ 이미 전형적 (베이스라인)

- SVEN / SafeCoder — prefix/instruction 튜닝으로 모델을 보안화. 데이터셋 중속·재학습 필요·제로데이 일반화 실패.
- Copilot Autofix — CodeQL 경고 + 코드흐름을 LLM 프롬프트로, *이미 상용 GA*.

★ 차별화 3대 메시지

- 재학습 없는 제로데이 대응 (GRASP 그래프 + IRIS taint 추론)
- 사후 스캔이 아닌 생성 중 개입 (SemGuard)
- 오탐 최소화된 신뢰 경고 (멀티에이전트 + 형식검증)

L2 뉴로심볼릭 정적분석 — IRIS

LLM이 프로젝트별 taint source/sink 명세를 추론 → CodeQL에 주입해 whole-repo 분석. CodeQL 단독 27건 → 55건(+28), 오탐률도 개선. (ICLR 2025)

L1 보안 지식그래프 추론 — GRASP

OWASP 실무 28개를 DAG로 구조화, 관련성 스코어링·의존순서 순회로 반복 리팩터링. Claude 0.59→0.82, 미지의 CVE서도 0.64(PromSec 0.28의 2배+). 재학습 불필요.

L0 생성 중 실시간 검증 — SemGuard

디코딩에 임베딩된 경량 평가기가 라인 단위 의미·보안 이탈을 감지, 오염 라인 이전으로 백트래킹. 취약 패턴 전파 자체를 차단.

L3 커버리지-가이드 LLM 퍼징

Fuzz4All(평균 커버리지 +36.8%, 98버그), CovRL(RL 커버리지 보상). Big Sleep 교환: 오픈엔드보다 변종분석(과거 패치를 시드로)이 현 LLM에 적합 — 실제 SQLite 제로데이 발견.

L4 적대적 멀티에이전트 검증

주장↔반박↔심판 "모의법정" 구조로 확정된 것만 사용자에게 노출. 레드/블루 프레임워크: 고유버그 +47%, 재발 -33%. 바이브코더의 오탐 무시 문제 해결.

L5 형식검증 — VeriGuard

인증·권한·결제 등 critical 경로 한정. 사용자 의도→안전 명세→정책 합성→형식검증으로 준수 증명. "아마 안전"이 아닌 증명 가능한 보장 → 엔터프라이즈 세일즈 포인트.

04 실패모드 → 보안 검증 기법 매핑

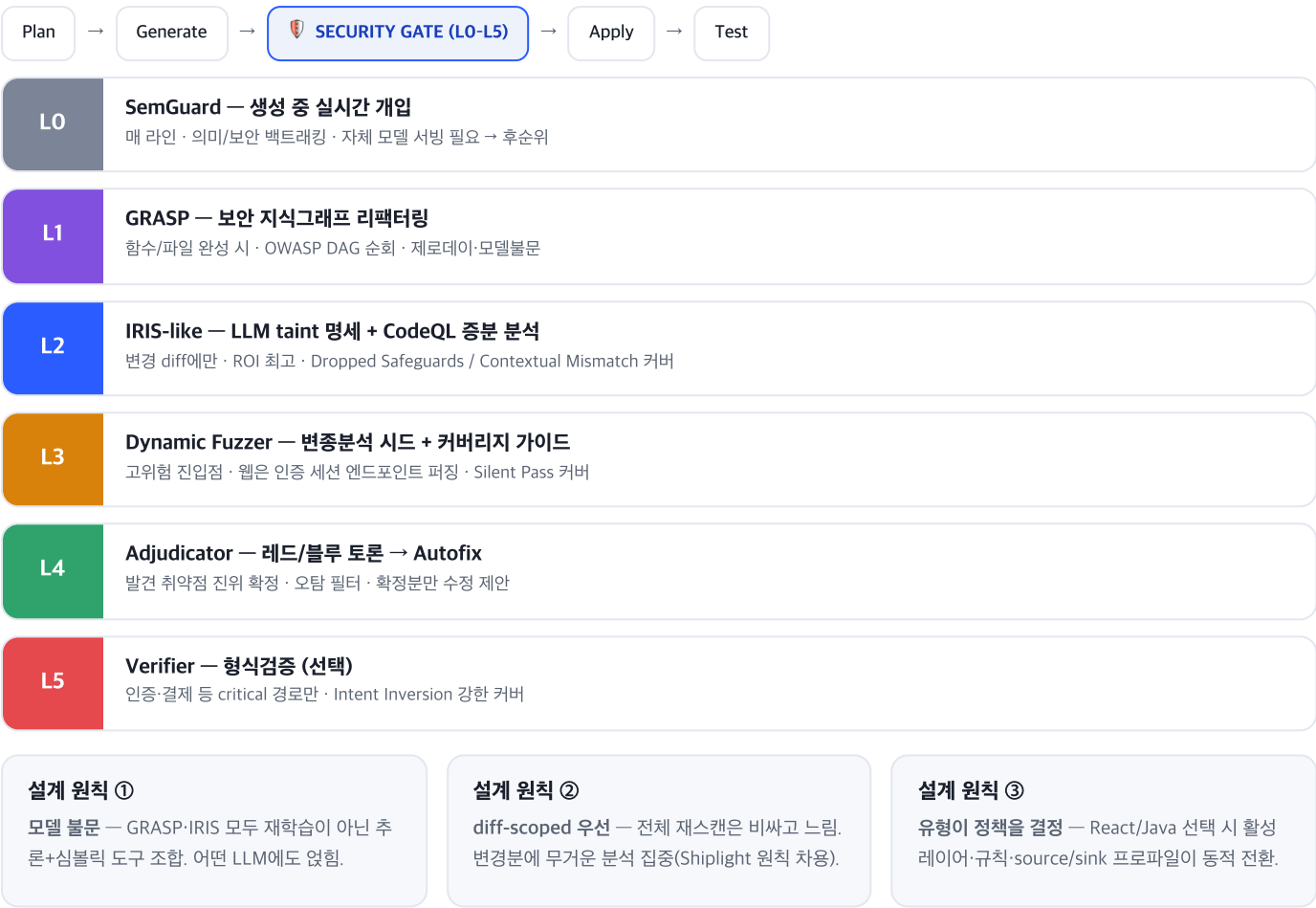
Shiplight의 4가지 실패모드를 보안 관점으로 재해석하면, 각기 다른 검증 레이어가 필요하다 → 단일 스캐너가 아닌 **tiered** 파이프라인이 정당화된다.

실패모드	보안적 재해석	왜 일반 스캐너로 못 잡나	담당 기법	위치
Intent Inversion 요구와 반대로 동작	인가 조건 반전(if !isAdmin→if isAdmin), 검증 로직 부정, "삭제 금지"인데 삭제	문법상 정상이라 SAST가 취약점으로 안 봄. 의도(spec) 대비 위반 이라 명세 필요	VeriGuard 형식검증 — 의도에서 안전 명세 추출·정책 준수 증명	L5 증명
Dropped Safeguards 방어 로직 조용히 삭제	리팩터링 중 입력검증·CSRF·파라미터 바인딩·rate-limit 소실	전체 스냅샷 스캔은 "원래 없음"과 "지워짐"을 구분 못 함 → diff 기준 회귀 탐지 필요	IRIS 증분 taint 분석 + 이전 커밋 방어 로직 존재 대조	L2 diff
Contextual Mismatch 그럴듯하나 맥락에 틀림	이 프레임워크/버전에 안 맞는 관용구 (deprecated 암호 API, ORM raw query 혼용)	범용 규칙셋은 프로젝트 고유 source/sink·버전 컨텍스트를 모름	GRASP 지식그래프 + IRIS 프로젝트별 taint 명세 추론	L1 커밋전
Silent Pass 테스트 통과하나 취약	정상 입력엔 통과, 악성 입력에 취약 (SQLi/XSS/path traversal)	개발자의 해피패스 테스트는 공격 입력을 넣지 않음	커버리지-가이드 LLM 퍼징 + 변종분석 시드(Big Sleep식)	L3 동적
(공통) 오탐 억제	위 4개에서 나온 경고의 진위 확정	단일 LLM 검증은 오탐↑ → 바이브코더가 무시하게 됨	적대적 멀티에이전트 (주장↔반박↔심판) → Autofix식 수정	L4 확정

표의 메시지: 4가지 실패모드가 서로 다른 검증 레이어를 요구한다. 그래서 단일 스캐너로는 부족하고, 계층형 파이프라인이 필요하다.

05 아키텍처 — Tiered Security Gate

에이전트 개발 루프의 생성 직후에 보안 게이트를 삽입. 비용/지연에 따라 계층을 나눠 변경 diff에 무거운 분석을 집중한다.



06 핵심 모듈

모듈	역할	기반 기술
Harness Core	계획·생성·적용·테스트 루프 관리, 보안 게이트 호출 지점 정의	Claude Agent SDK 등
Security Orchestrator	프로젝트 유형→활성 레이어 결정, 티어링/예산 관리, 결과 병합	자체 구현
Policy Profiles	유형별(React/Node/Java) source·sink·규칙·CWE 프로파일	Project CodeGuard 부트스트랩
L1 GRASP Engine	OWASP 실무 DAG 순회·관련성 스코어링·반복 리팩터링	그래프 + LLM
L2 Static (IRIS-like)	LLM taint 명세 추론 → CodeQL 증분 쿼리	CodeQL + LLM
L3 Dynamic Fuzzer	변종분석 시드 + 커버리지 가이드 퍼징(웹=요청 퍼징)	OSS-Fuzz-Gen/Fuzz4All 아이디어
L4 Adjudicator	레드/블루 토론으로 오탐 필터, 확정분 Autofix	멀티에이전트
L5 Verifier	critical 경로 형식검증(선택)	VeriGuard식
Findings / Audit	결과 저장 + "왜 이 판정인지" 근거(그래프 경로) 기록	DB + 리포트
Eval Harness	자사 효과 정량 측정	SecureVibeBench, CWE-Bench-Java

07 개발 로드맵

- Phase 0 — **빠대 + 평가 먼저** · 2~3주

 - 에이전트 루프에 "보안 게이트 훅"(no-op) 삽입 → 생성 후 diff 가로채는 지점 확보
 - 평가 하네스 먼저 세팅(SecureVibeBench/CWE-Bench-Java) — baseline 0일부터 측정
 - 프로젝트 유형 감지 + Policy Profile 로더(React·Node 1종)
- Phase 1 — **MVP: 정적 중심 게이트** · 3~5주

 - L2 (IRIS-like)부터 — ROI 최고·근거 강함(+28건 실측)
 - L4 축소판 — 단일 반박 에이전트로 오탐만 걸러도 체감 큼
 - 커버: Dropped Safeguards + Contextual Mismatch → 데모 가능
- Phase 2 — **생성 중 개입 + 지식그래프** · 4~6주

 - L1 GRASP — 제로데이·모델불문 세일즈 포인트 확보
 - L0 SemGuard(선택) — 인프라 부담 크므로 후순위
- Phase 3 — **동적 검증** · 5~8주

 - L3 Fuzzing — 인증 세션 엔드포인트 퍼징 + 변종분석 시드 → Silent Pass 커버
 - Shiplight(기능) vs 우리(보안) 상보 포지셔닝 완성
- Phase 4 — **엔터프라이즈** · 이후

 - L5 VeriGuard — 인증·결제 형식검증 → Intent Inversion 커버
 - SBOM/공급망(Project CodeGuard), 감사 리포트

08 착수 전 결정해야 할 3가지

이후 구조를 크게 가르는 분기점. 권장 기본값을 함께 표기했다.

① 하네스

새로 제작 vs 기존 에이전트(Claude Code) 위에 얹기

▶ 권장: 얹기 (플러그인/혹으로 MVP 가속)

② 대상 스택

다중 스택 vs 한 스택 우선

▶ 권장: React + Node 우선 (완성도 먼저)

③ 모델 서빙

자체 서빙(L0 가능) vs API 모델만

▶ 권장: API 모델 + L1·L2 중심 MVP

권장 MVP 한 줄 요약: 기존 에이전트에 혹으로 얹고 · React+Node로 좁히고 · API 모델 + L2(IRIS식 diff 정적분석) + L4(오탐 필터)로 시작 → 평가 하네스로 "취약 검출 +N건, 오탐 -M%"를 숫자로 증명.

09 참고 문헌

GRASP — Graph-Based Reasoning for Secure Code Generation, arXiv 2510.09682

IRIS — LLM-Assisted Static Analysis, ICLR 2025 / arXiv 2405.17238

Project Zero — From Naptime to Big Sleep, Google (2024.10)

Fuzz4All — Universal Fuzzing with LLMs / OSS-Fuzz-Gen

CovRL — Coverage-guided RL for LLM Mutation, arXiv 2402.12222

SemGuard — Real-Time Semantic Evaluator, arXiv 2509.24507

VeriGuard — Verified Code Generation for Agent Safety, arXiv 2510.05156

Refute-or-Promote — Adversarial Multi-Agent Review, arXiv 2604.19049

Copilot Autofix — Fixing Vulnerabilities with AI, GitHub Engineering Blog

SVEN — Security Hardening & Adversarial Testing, arXiv 2302.05319

Asleep at the Keyboard — Copilot Security, IEEE S&P 2022

SecureVibeBench — Benchmarking Secure Vibe Coding, arXiv 2509.22097

Adversarial Prompts vs Secure Code Gen, arXiv 2601.07084

Project CodeGuard — COSAI/OASIS Secure-by-default Ruleset

Shiplight — AI-Generated Code Has More Bugs (시장 근거)

AI Secure Dev Tool · 기술 차별화 & 개발 계획 · 2026-07-20 — 내부 검토용. 시장 벤더 수치(CodeRabbit/Shiplight)는 동기부여용이며, 정량 주장은 피어리뷰 자료와 병기 권장.